

Caleb AI Multi Model Orchestration with Hollows and Role Rotation

Executive summary and key findings

Original Caleb AI design claim. In the user-provided Caleb AI brief, *Hollows* are the non-LLM execution units that take over mundane, logic-driven, deterministic work locally, while the model layer stays focused on reasoning, planning, interpretation, and synthesis. The same brief also establishes *role rotation* as the mechanism for escalating difficult reasoning into differentiated cognitive roles rather than leaving all work inside a single undifferentiated model pass. [filecite:turn0file0](#) [filecite:turn0file1](#)

The strongest source-backed interpretation of that design is technically sound: Caleb AI should be structured as an **Orchestration Core** that routes tasks between an **adaptive model layer** and a **deterministic Hollow layer**, with a **Verified Return Path** that refuses to let raw tool output become trusted state until it has passed schema validation, policy checks, provenance stamping, and replayability or reproducibility checks. This design is strongly aligned with the direction of modern tool calling, structured outputs, provenance standards, AI governance guidance, and local-first architecture. OpenAI and Anthropic both document typed tool/function calling as the standard pattern for models to trigger external execution; JSON Schema gives the contract discipline for those interfaces; MCP gives a standard message-oriented integration model; W3C PROV-O gives the vocabulary for provenance; NIST AI RMF anchors governance and trustworthiness; OWASP GenAI guidance anchors the major security risks; and local-first architecture provides a strong rationale for pushing bounded deterministic workloads closer to the user and their data.

1

The evidence also supports the central Hollow thesis directly. Toolformer argues that language models remain weak at “basic functionality” such as arithmetic and factual lookup, where much simpler external components can do better; ReAct shows that interleaving reasoning with actions against an environment improves both results and interpretability; and Anthropic’s production guidance recommends using the simplest pattern possible first, only adding complexity when the task actually warrants it. In practical terms, that means Caleb AI should not ask an expensive reasoning model to do what a deterministic Hollow can do faster, cheaper, and more reliably. [2](#)

Role rotation is best treated as a **conditional escalation policy**, not the default path. Source-backed patterns support planner/executor decomposition, evaluator-optimizer loops, reflection, and multi-agent debate, and multiple papers show that these patterns can improve reasoning or factuality on hard tasks. At the same time, recent evidence also shows that added agent complexity can produce diminishing returns, that majority voting sometimes explains most of the measured gains attributed to debate, and that token costs rise quickly with extra roles and rounds. The right Caleb AI position is therefore: **default to single-pass or simple workflows, escalate to role rotation only when ambiguity, stakes, multi-step dependency, or contradiction risk crosses a threshold.** [3](#)

The most defensible professional-grade architecture for Caleb AI is therefore this: **models think, Hollows work, Caleb orchestrates, and the Verified Return Path decides what is trustworthy enough to circulate, store, or act upon.** The rest of this report turns that principle into a paper-ready architecture, a slide-ready diagram set, and an implementation roadmap. [filecite\turn0file0](#) [filecite\turn0file1](#)

Source backed architectural foundations

Original Caleb AI design claim. The names *Caleb AI*, *Hollows*, *Hollow Server*, *Verified Return Path*, and the Planner → Analyst → Critic → Synthesizer rotation are preserved exactly as requested. They are treated here as proprietary Caleb AI concepts rather than borrowed industry terms. [filecite\turn0file0](#)
[filecite\turn0file1](#)

Source-backed implementation parallel. The closest existing technical analog to a Hollow is a strictly typed external tool/function invoked by a model but executed outside the model. OpenAI documents function calling/tool calling as a way for models to interface with external systems and application-side actions, with tools defined by JSON Schema and a five-step execution loop that includes model request, tool call, application execution, tool result, and final model response. Anthropic documents the same basic separation, distinguishing client-side tools that execute in the application from server-side tools that execute on Anthropic infrastructure. [4](#)

That matters because typed external execution is not just a convenience; it is the mechanism that allows Caleb AI to enforce **hard boundaries** between reasoning and execution. OpenAI documents strict mode as a way to ensure function calls reliably adhere to the declared schema rather than behaving “best effort,” and Anthropic documents `strict: true` for schema-exact tool calls. JSON Schema itself defines a mature vocabulary for structural validation, including object, array, numeric, string, and conditional constraints, and it also defines structured validation outputs with a required boolean `valid` field and optional `errors` and `annotations`. A professional Caleb AI implementation should anchor Hollow contracts in JSON Schema 2020-12 and make strict validation non-optional for trusted paths. [5](#)

Caleb AI also maps cleanly onto the strong distinction Anthropic draws between **workflows** and **agents**. Anthropic defines workflows as systems where LLMs and tools are orchestrated through predefined code paths, while agents dynamically direct their own process and tool use. Hollows fit the workflow side of that spectrum: bounded, predictable, deterministic, and inspectable. Role rotation fits the agentic side: dynamic reasoning decomposition when a task is too ambiguous or open-ended to hardcode. Anthropic’s guidance that developers should start with the simplest solution and only add complexity when it clearly improves outcomes is exactly the right operational principle for Caleb AI. [6](#)

The provenance and trust side of the architecture is equally well grounded. W3C PROV-O defines a foundation for representing provenance across systems, using **Entities**, **Activities**, and **Agents**, together with relations such as `wasGeneratedBy`, `used`, `wasAssociatedWith`, and `wasInformedBy`. For Caleb AI, a Hollow run is an Activity, its input and output artifacts are Entities, and the Orchestration Core, model provider, end user, or Hollow runtime are Agents. That gives Caleb AI a standards-based ledger design rather than an ad hoc logging story. [7](#)

NIST AI RMF provides the governance frame that keeps this from becoming an elegant but unsafe architecture. NIST defines trustworthy AI characteristics as valid and reliable, safe, secure and resilient,

accountable and transparent, explainable and interpretable, privacy-enhanced, and fair with harmful bias managed. It also describes GOVERN as a cross-cutting function that should inform MAP, MEASURE, and MANAGE, and explicitly treats separating builders/users from validators/verifiers as a best practice. Caleb AI's Hollow verifier path and role-separated reasoning are directly compatible with that guidance. ⁸

On the security side, OWASP's GenAI guidance is directly relevant because Caleb AI is, in effect, a guarded agent platform. OWASP's current Large Language Model Top 10 includes prompt injection, insecure output handling, model denial of service, sensitive information disclosure, insecure plugin design, excessive agency, overreliance, and model theft. MCP's own specification likewise warns that tool paths expose arbitrary data access and code execution concerns, and it requires explicit user consent, data privacy controls, and tool safety precautions. These sources together imply that **Hollows must be least-privilege, sandboxed, schema-validated, policy-gated, and never allowed to turn raw model suggestions directly into unsafe side effects.** ⁹

The local-first aspect of the design is also more than aesthetic. Ink & Switch's local-first architecture argues that keeping the primary copy of data on the local device can improve ownership, privacy, offline operation, and responsiveness, precisely because the user does not have to wait for every operation to round-trip through a remote server. Caleb AI's Hollow Server is a strong fit for that model: deterministic work, sensitive data handling, and repeatable transformations should happen as close to the data as possible, with cloud model calls reserved for the parts that actually need model intelligence. ¹⁰

System specification and analytical report

Original Caleb AI design claim. The Caleb AI architecture should be divided into five conceptual planes: **Orchestration Core, Model API Layer, Communication Bus, Hollow Server, and Verified Return Path.** This is the cleanest way to preserve the distinction the user wants between reasoning and deterministic work. ¹¹

Orchestration Core. The Orchestration Core is the policy brain of the whole system. It does not do the reasoning itself and it does not do the deterministic work itself. Instead, it classifies the task, determines whether the request is simple, deterministic, ambiguous, high-stakes, or multi-stage, selects a workflow pattern, chooses a model or role set, allocates Hollow calls, and decides whether the result can exit through the Verified Return Path. This is consistent with research arguing that orchestrated systems are easier to debug and test than highly choreographed distributed interactions, and with Anthropic's distinction between routing, orchestrator-workers, and evaluator-optimizer patterns. ¹¹

Model API Layer. The Model API Layer is Caleb AI's adaptive reasoning surface. It talks to one or more model providers through typed or normalized interfaces, manages prompt templates, controls tool exposure, tracks costs, and enforces role prompts. OpenAI explicitly recommends limiting the number of functions exposed up front, using tool search to defer large or infrequently used parts of the tool surface, and recognizing that tool definitions count against context and billing. Anthropic similarly notes that the `tools` parameter, tool schemas, and tool result blocks all add token overhead. That means the Model API Layer should expose only the few Hollows relevant to the current turn rather than dumping the full Hollow registry into every prompt. ¹²

Communication Bus. The Communication Bus should be platform-agnostic but strict in structure. A sound design is a typed event envelope carrying `trace_id`, `span_id`, `parent_span_id`, `run_id`, `task_id`, `actor_id`, `role_id`, `hollow_id`, `schema_version`, `policy_tags`, `input_hash`, `output_hash`, `status`, and timestamps. MCP shows that JSON-RPC 2.0 plus stateful connections, capability negotiation, progress tracking, cancellation, error reporting, and logging is a viable standard pattern for model-adjacent systems. OpenTelemetry provides the trace vocabulary for distributed requests, where traces represent the path of a request and spans capture nested units of work, including attributes, events, and causal links. Caleb AI therefore has a clear source-backed basis for a typed, traceable bus even without choosing a specific broker technology. ¹³

Hollow Server. The Hollow Server is Caleb AI's deterministic execution engine. It should host a registry of Hollows, a scheduler, a runtime sandbox, a validator, a policy engine, a chain executor, and a provenance emitter. A Hollow should be small, typed, testable, deterministic within its declared boundaries, and versioned as a public API surface. SemVer is the right versioning discipline here: incompatible changes increment MAJOR, backward-compatible additions increment MINOR, and backward-compatible fixes increment PATCH. That matters because Hollow contracts are application interfaces, and version ambiguity is one of the fastest ways to destroy replayability and trust. ¹⁴

Hollow taxonomy. A comprehensive preliminary taxonomy for Caleb AI includes: **compute Hollows** for arithmetic, date math, scoring, and threshold logic; **validation Hollows** for schema validation, rule checks, and contradiction scans; **transformation Hollows** for normalization, parsing, tabular shaping, and packaging; **policy Hollows** for allow/deny gating and side-effect control; **provenance Hollows** for hashing, stamping, signing, and ledger writing; **routing Hollows** for deterministic triage; and **recovery Hollows** for retry/backoff and dead-letter handling. This taxonomy is an original Caleb AI proposal, but it is motivated by the role separation in tool-calling systems and the workflow patterns Anthropic documents for routing, prompt chaining, and orchestrator-workers. ¹⁵

Hollow lifecycle. A production Hollow should move through a disciplined lifecycle: **Registered**, **Approved**, **Versioned**, **Callable**, **Running**, **Validating**, **Verified** or **Rejected**, then optionally **Retried**, **Quarantined**, **Deprecated**, or **Retired**. The reason to formalize this is simple: NIST stresses governance as a cross-cutting function, JSON Schema supports machine-checkable validity, and workflow research shows that reliable execution requires explicit persistence, logs, and recovery behavior. A Hollow that has no state model is not a production component; it is just a function. ¹⁶

Original Caleb AI design claim. The Hollow contract should be the canonical trust boundary in Caleb AI. The following JSON example is a recommended contract instance for a Hollow run.

```
{
  "contract_version": "1.0.0",
  "hollow_id": "hollow.schema_validate",
  "run_id": "run_01JZC8Z6M0S4Y2QJ7X4M8W2G9A",
  "trace_id": "trace_01JZC8YV2D0CBEM9V9D9E1H0NT",
  "task_id": "task_01JZC8YQ2ATX9QH5Q4Q4H4RC5Q",
  "intent": "validate_candidate_output",
  "determinism_mode": "strict",
  "inputs": {
```

```

    "schema_ref": "schema://caleb/hollow_result/1.0.0",
    "candidate_ref": "artifact://candidate/output.json"
  },
  "verification": {
    "input_hash": "sha256:3d5d...a2",
    "runtime_digest": "sha256:9aa1...42",
    "schema_valid": true,
    "policy_pass": true,
    "reproducible": true
  },
  "output": {
    "status": "verified",
    "trust_tier": "T2",
    "artifact_ref": "artifact://verified/output.json"
  },
  "provenance": {
    "activity_id": "prov:activity:run_01JZC8Z6M0S4Y2QJ7X4M8W2G9A",
    "used": [
      "artifact://candidate/output.json"
    ],
    "generated": [
      "artifact://verified/output.json"
    ]
  },
  "return_path": {
    "allow_model_consume": true,
    "allow_side_effect": false,
    "allow_persist_as_truth": true
  }
}

```

Source-backed implementation parallel. This contract style is strongly supported by OpenAI’s and Anthropic’s strict tool-use guidance and by JSON Schema’s structural validation model. OpenAI recommends always enabling strict mode for function calls; Anthropic documents `strict: true` for schema-exact tool calls; and JSON Schema defines both structural validation keywords and standardized output structures for validity, errors, and annotations. ¹⁷

Verified Return Path. The Verified Return Path is the deciding checkpoint between “a component produced output” and “Caleb AI is allowed to trust, store, cite, or act on that output.” This is one of the most important original Caleb AI ideas in the brief, and it should be implemented as a multi-gate pipeline: schema validation, deterministic replay or hash confirmation, policy evaluation, provenance emission, contradiction or conflict scan, and trust tier assignment. This design is justified by OWASP’s concerns about insecure output handling and excessive agency, MCP’s explicit tool-safety requirements, and the broader NIST requirement for accountable, transparent, safe, and secure systems. ¹⁸

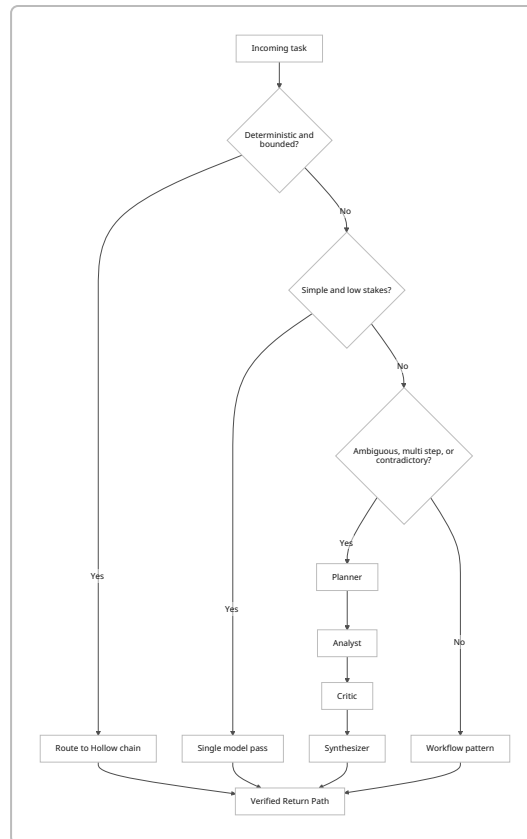
Hollow to Hollow chaining. Hollow-to-Hollow chaining should happen inside the Hollow Server as a typed DAG, not by repeatedly bouncing back through the model layer unless the task truly needs model judgment

between steps. Anthropic's routing, prompt chaining, and orchestrator-workers patterns all support decomposing work into specialized stages, and workflow engine research supports explicit orchestration for easier debugging and recovery. In Caleb AI terms, the model should not micromanage every deterministic edge in the graph; instead, it should describe intent, and the Orchestration Core should launch a known chain of Hollows where possible. ¹⁹

Role rotation logic. The proposed Planner → Analyst → Critic → Synthesizer sequence is an original Caleb AI role model, but it has strong support from adjacent research. Plan-and-Solve supports explicit planning before execution to reduce missing-step errors. Self-Refine supports iterative feedback and refinement loops. Reflexion supports feedback-driven memory and better downstream decisions. Anthropic's evaluator-optimizer pattern explicitly describes one model producing and another evaluating in a loop, and their orchestrator-workers pattern matches the idea of a central planner delegating specialized work. The best interpretation is:

- **Planner** defines the task graph, scope, decomposition, required evidence, and which Hollows or models should be used.
- **Analyst** gathers and structures evidence, runs or requests Hollows, and constructs candidate findings.
- **Critic** stress-tests assumptions, contradiction points, unsafe paths, schema failures, missing evidence, and overreach.
- **Synthesizer** produces the final integrated response, explicitly marked by trust tier and supported artifacts.

That is superior to a monolithic single-pass chat pattern when the task has meaningful decomposition, multiple error modes, or a need for internal adversarial checking. But it is not universally superior: Anthropic recommends simplest-first design, and recent work finds that majority voting often captures much of the gain attributed to debate while being more reliable and cheaper. Caleb AI should therefore use role rotation selectively, not reflexively. ²⁰



Trust hierarchy. A practical Caleb AI trust hierarchy should rank outputs approximately as follows: **T0 raw model suggestion**, **T1 schema-valid but unverified output**, **T2 deterministic Hollow output reproduced or hash-confirmed**, **T3 deterministic output plus policy clearance and provenance anchoring**, and **T4 human-approved or externally authoritative result with full citation/provenance chain**. This exact tiering is an original recommendation, but it is grounded in NIST’s emphasis on trustworthiness and accountability, JSON Schema’s machine-verifiable validation, PROV-O’s provenance relations, and MCP/OWASP security constraints. ²¹

Provenance and ledger design. The ledger should be append-only and store at least the Hollow or role activity, inputs used, outputs generated, who or what agent performed the step, timestamps, policy result, and hashes. A clean PROV mapping is: input/output artifacts as `prov:Entity`, Hollow or model invocation as `prov:Activity`, Caleb AI core, model provider, user, or runtime as `prov:Agent`, with `prov:used`, `prov:wasGeneratedBy`, `prov:wasAssociatedWith`, and `prov:wasInformedBy` linking them. OpenTelemetry spans should mirror the same causal tree in runtime telemetry so engineering and governance views line up. ²²

Security model. Caleb AI should assume that all model suggestions are advisory, all tool arguments are untrusted until validated, and all tool outputs are tainted until they pass the Verified Return Path. User consent, explicit data-sharing boundaries, and clear action authorization are required, as MCP’s security model states. OWASP’s current LLM risks imply concrete controls: prompt injection defenses, output validation, side-effect gating, least-privilege plugin/tool design, limits on autonomous action, and explicit checks against overreliance. In practical terms, that means sandbox all Hollows, separate read-only from side-effect-capable Hollows, scope every secret to the minimum feasible capability, require policy approval

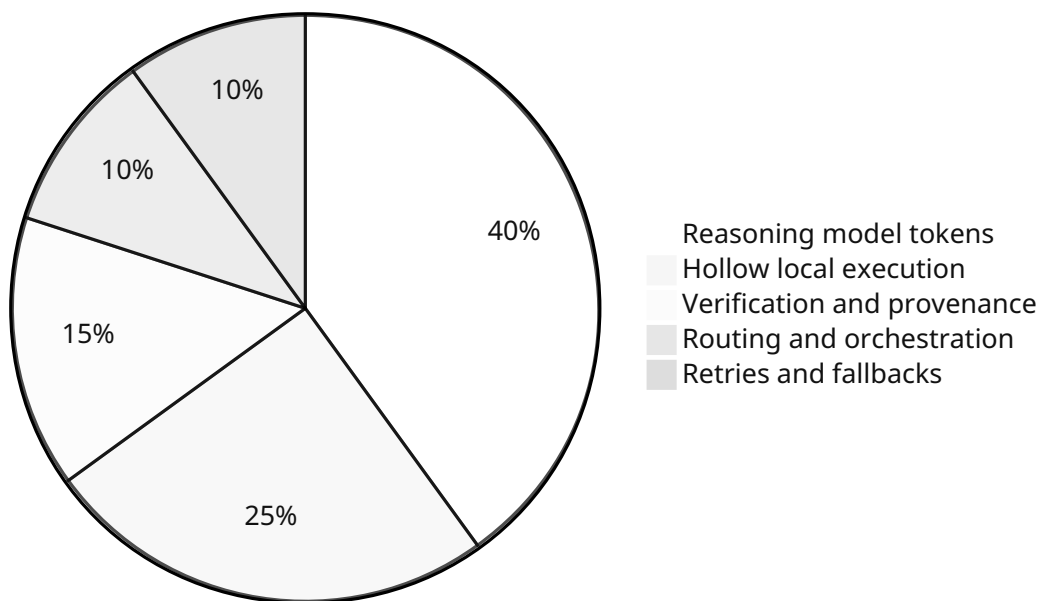
before actuation, and quarantine any Hollow whose runtime digest, schema version, or result signature deviates from accepted expectations. ²³

Versioning and governance. Hollow APIs, manifests, trust logic, and role prompts should all be versioned. SemVer should govern public Hollow interfaces and manifest evolution; NIST GOVERN should govern who may approve a new Hollow, who can raise its trust tier, who can approve a side-effect capability, and who can deprecate or quarantine it. This is especially important because strong version discipline is what makes replay, audit, and backward compatibility practical over time. ²⁴

Testing strategy. Caleb AI should use four test layers: contract tests for every Hollow and schema, replay tests for every critical orchestration path, adversarial tests for prompt injection and unsafe side effects, and task-level evaluation for role rotation thresholds. Anthropic explicitly recommends extensive testing in sandboxed environments and measurable iteration; NIST highlights verification and validation role separation; and workflow-engine research suggests orchestrated systems are easier to debug and test than loosely choreographed interactions. ²⁵

Cost and performance model. OpenAI's guidance is explicit: reducing requests, minimizing tokens, and selecting smaller models reduce both latency and cost. OpenAI also notes that loaded function definitions count toward context and billing; Anthropic similarly notes that tools, schemas, tool use blocks, and tool results add input-token overhead. This makes the economic case for Hollows especially strong: when a task is deterministic, the model should produce the *decision to call a Hollow*, not the deterministic work itself. Role rotation should be triggered only where the expected reduction in error, hallucination, or rework exceeds the added model cost. Local-first Hollow execution then adds a second performance advantage by removing remote latency for many bounded operations. ²⁶

Caleb AI runtime cost allocation after Hollow refactor



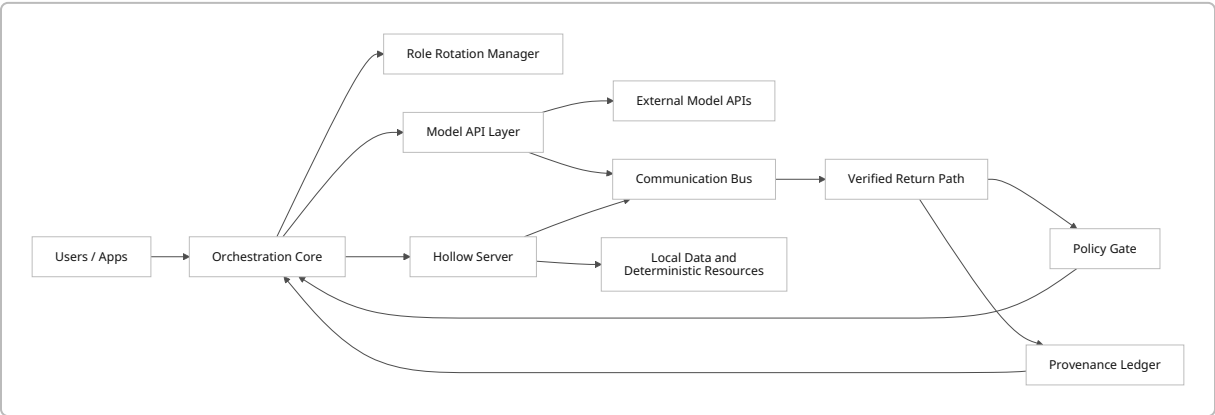
The chart above is **illustrative rather than benchmarked**. The source-backed point is the direction of travel: fewer requests and fewer tokens lower cost, while excessive tool surface area and unnecessary multi-role loops raise it. ²⁷

Vector ready diagram specifications

The diagrams below are specified for **professional slide and paper use**. A consistent visual system is recommended across the whole set: **Inter** or **IBM Plex Sans** for labels, **IBM Plex Mono** for IDs and hashes, 1.5–2 pt strokes, 8 px corner radius, white background, neutral gray bus connections, blue/purple for model cognition, teal for Hollows, green for verification/trust, amber for warning states, and red only for failure or quarantine states. Export **SVG as the master**, plus **PNG at 2×** for slides and **PDF** for papers or handouts. The architectural substance of these diagram specs is sourced from the tool-calling, provenance, governance, and security materials discussed above. ²⁸

Overall system

Purpose: show the full Caleb AI control plane at a glance.
Audience: executives, architects, investors, engineering leadership.
Elements: user/app entry, Orchestration Core, Role Rotation Manager, Model API Layer, Communication Bus, Hollow Server, Verified Return Path, Provenance Ledger, Policy Gate, external model APIs, local data/resources.
Layout notes: left-to-right or top-to-bottom; keep Hollows and models visibly separated by a vertical boundary.
Color/typography: model blocks in indigo, Hollow blocks in teal, trust/ledger in green, transport in gray.
Export: SVG master, PNG 2×, PDF.



```
[Users] -> [Orchestration Core]
      |-> [Role Rotation Manager]
      |-> [Model API Layer] -> [External Models]
      |-> [Hollow Server] -> [Local Data]
[Model API Layer] ----\
                      -> [Communication Bus] -> [Verified Return Path]
```

[Hollow Server] -----/

|-> [Policy Gate]

\-> [Provenance Ledger]

Hollow Server architecture

Purpose: explain the internal anatomy of the Hollow layer.

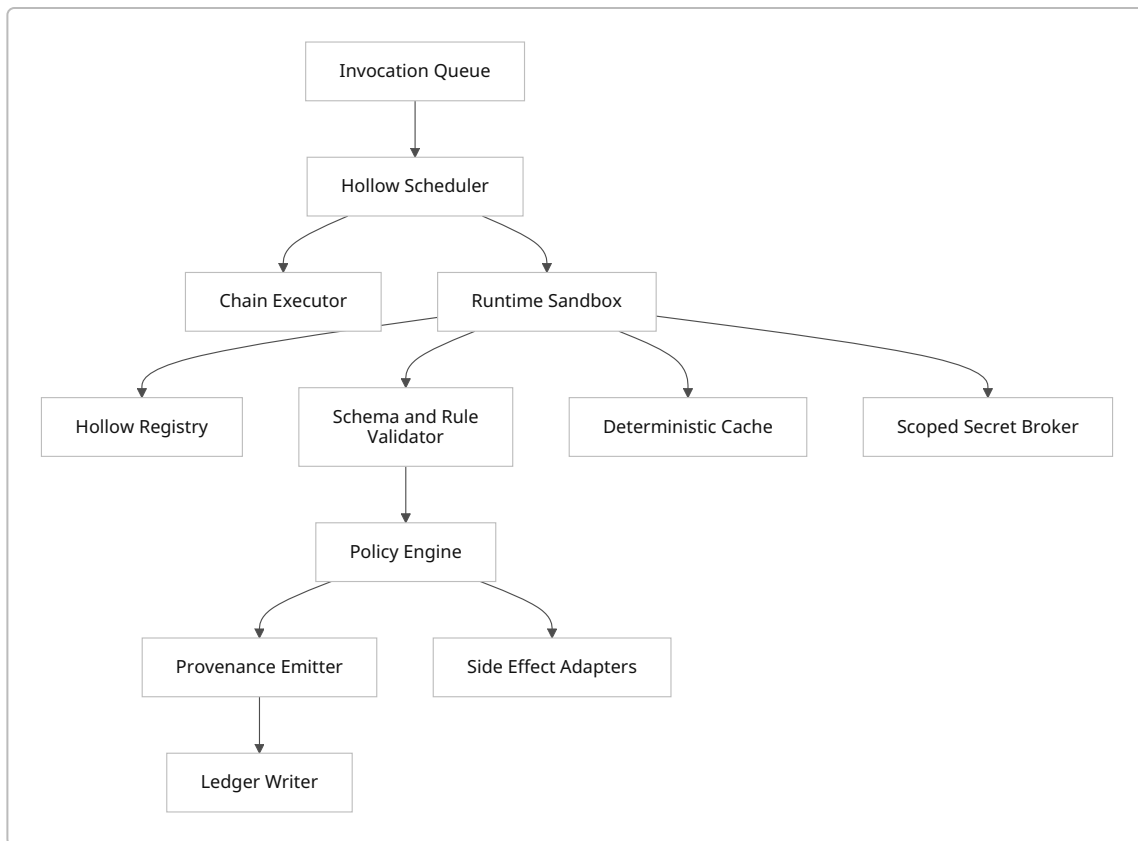
Audience: backend engineers, platform engineers, security reviewers.

Elements: Hollow registry, scheduler, runtime sandbox, chain executor, validator, cache, secret broker, policy engine, provenance emitter, side-effect adapter lane.

Layout notes: centralize scheduler and validator; place side-effect adapters behind a hard policy boundary.

Color/typography: Hollow internals teal; policy/validator green; side-effect path amber or red-framed.

Export: SVG master, PNG 2x, PDF.



[Queue] -> [Scheduler] -> [Chain Executor]

\-> [Runtime Sandbox]

|-> [Registry]

|-> [Validator] -> [Policy Engine] -> [Provenance] ->

[Ledger]

|-> [Cache]

|-> [Scoped Secrets]

\-> [Side Effect Adapters]

Hollow invocation sequence

Purpose: show the exact request-response lifecycle from model decision to verified result.

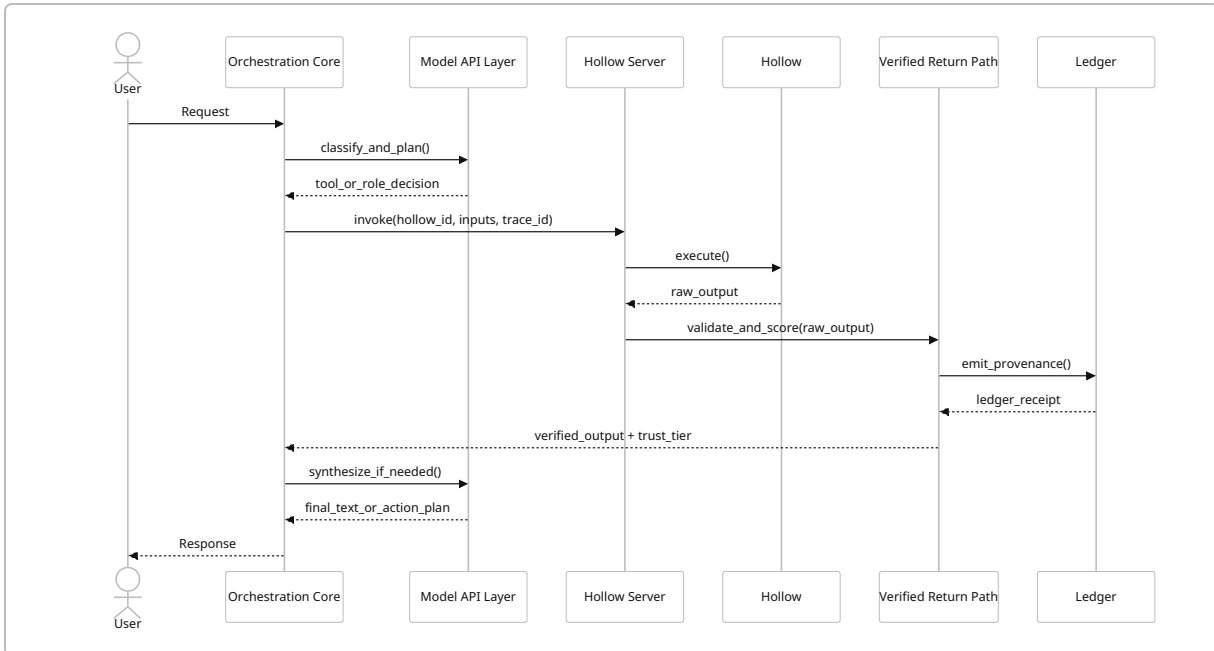
Audience: API engineers, solution architects, documentation writers.

Elements: user, Orchestration Core, model layer, Hollow Server, specific Hollow, validator, ledger, final synthesizer.

Layout notes: use sequence diagram; keep the second model call visually distinct from the Hollow run.

Color/typography: model lifelines indigo, Hollow lifelines teal, verification lifelines green.

Export: SVG master, PNG 2x, PDF.



User -> Caleb -> Model decides -> Hollow invoked
Hollow runs -> Output validated -> Provenance written
Verified result returns -> Model synthesizes if needed -> User receives answer

Hollow lifecycle state machine

Purpose: document operational states for governance, deployment, and recovery.

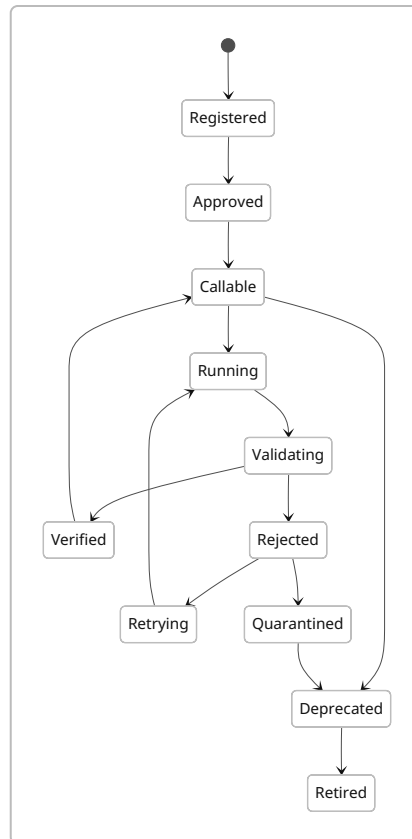
Audience: SRE, platform engineering, compliance, DevOps.

Elements: Registered, Approved, Callable, Running, Validating, Verified, Rejected, Retrying, Quarantined, Deprecated, Retired.

Layout notes: arrange success path on top, failure path below.

Color/typography: success in green/teal, quarantine in red, deprecated in gray.

Export: SVG master, PNG 2x, PDF.



Registered -> Approved -> Callable -> Running -> Validating -> Verified
 \-> Rejected -> Retrying -> Running
 \-> Rejected -> Quarantined ->
 Deprecated -> Retired

Hollow output trust pipeline

Purpose: make the Verified Return Path visually concrete.

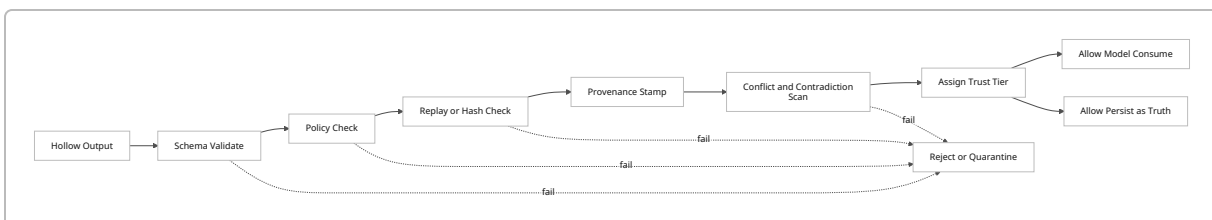
Audience: security, governance, engineering, auditors.

Elements: output receipt, schema validation, policy check, reproducibility/hash check, provenance emission, contradiction check, trust tier assignment, final routing.

Layout notes: linear pipeline with a visible reject lane.

Color/typography: all gate nodes green; reject lane red.

Export: SVG master, PNG 2x, PDF.



```

Output -> Schema -> Policy -> Replay/Hash -> Provenance -> Conflict Scan ->
Trust Tier
\_____failure ->
Reject/Quarantine

```

Hollow to Hollow chaining

Purpose: show how deterministic DAG execution bypasses unnecessary model hops.

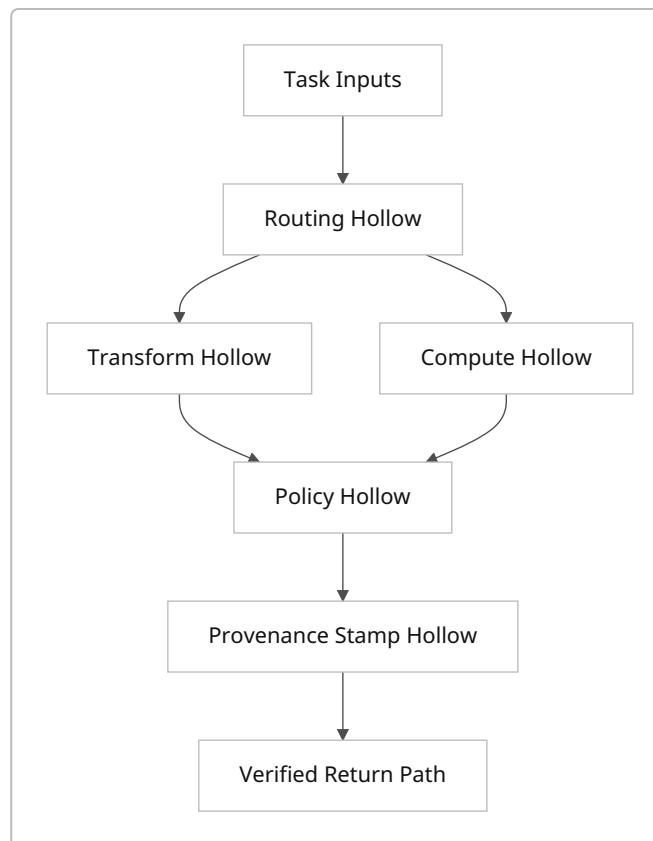
Audience: backend architects, workflow designers, performance engineers.

Elements: router Hollow, transform Hollow, compute Hollow, policy Hollow, provenance stamp, return path.

Layout notes: DAG with two branches joining at a policy or packaging node.

Color/typography: all Hollows teal; join node green.

Export: SVG master, PNG 2x, PDF.



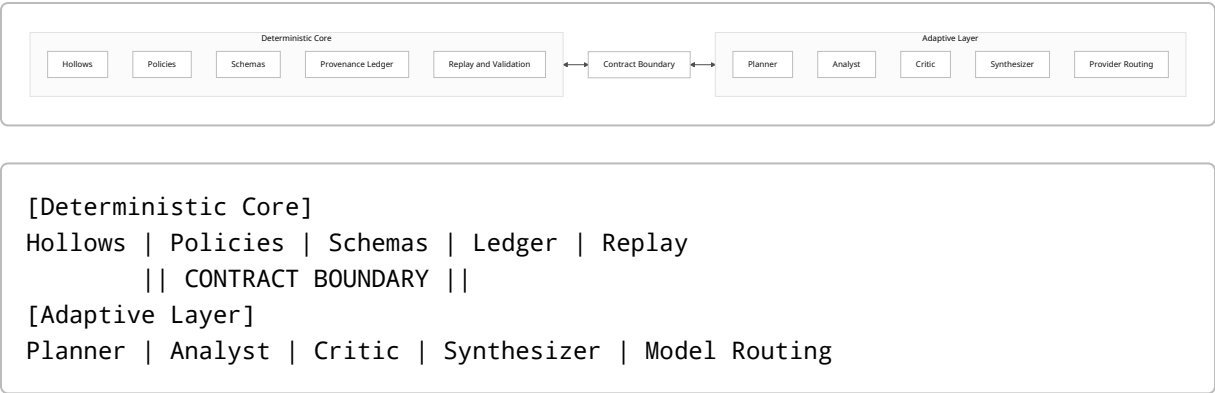
```

Inputs
-> Routing Hollow
  -> Transform Hollow --\
                                -> Policy Hollow -> Provenance Stamp -> Verified
Return
  -> Compute Hollow ----/

```

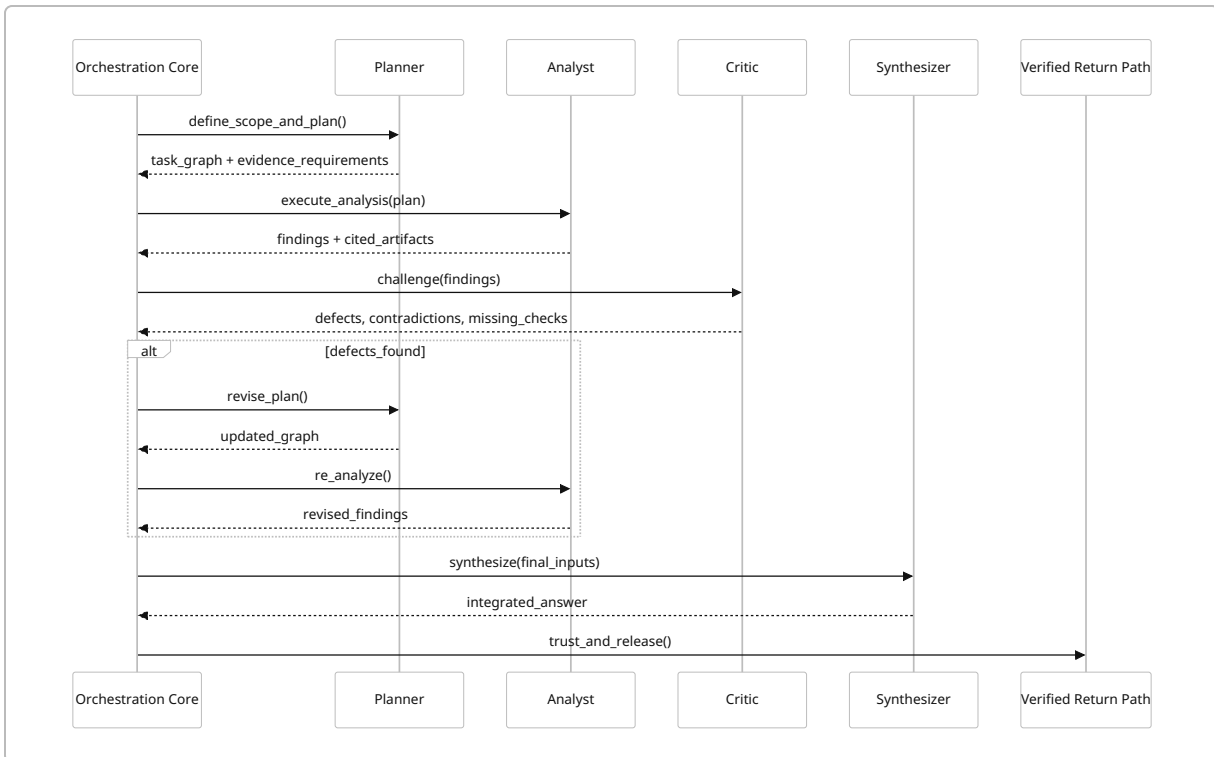
Deterministic core vs adaptive layer

Purpose: visually separate what Caleb AI should harden from what it should let remain model-adaptive.
Audience: strategy, architecture, product, governance.
Elements: left deterministic core; right adaptive layer; central contract boundary.
Layout notes: split screen diagram with a hard vertical divider labeled "contract boundary."
Color/typography: deterministic side teal/green; adaptive side indigo.
Export: SVG master, PNG 2x, PDF.



Role rotation flow

Purpose: show the main cognitive handoff across Planner, Analyst, Critic, and Synthesizer.
Audience: researchers, model engineers, product leads, reviewers.
Elements: task receipt, planning, evidence gathering, critique loop, synthesis, verification.
Layout notes: sequence diagram; include return arrows from Critic to Planner/Analyst if material defects appear.
Color/typography: role nodes in progressively darker indigo; verification in green.
Export: SVG master, PNG 2x, PDF.



Planner -> Analyst -> Critic -> Synthesizer

^

|_____||_____|| when contradictions or missing evidence appear

Final answer -> Verified Return Path

Role rotation logic tree

Purpose: define exactly when role rotation is warranted versus overkill.

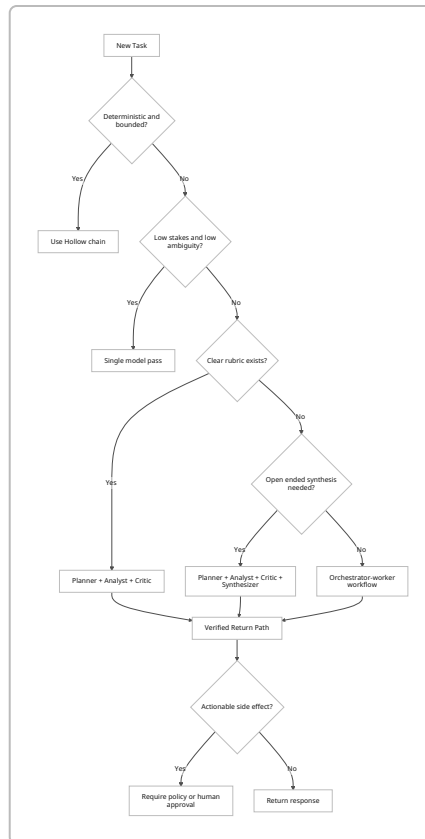
Audience: system designers, policy authors, optimization teams.

Elements: deterministic check, stakes check, ambiguity check, rubric availability, contradiction risk, model failure history, human approval branch.

Layout notes: start from "new task"; keep simple/single-path outcomes prominent.

Color/typography: default path gray/teal; escalation path indigo.

Export: SVG master, PNG 2x, PDF.



If deterministic -> Hollow
 Else if simple/low stakes -> single model
 Else if rubric/criteria exist -> Planner + Analyst + Critic
 Else if open-ended integration needed -> add Synthesizer
 If side effect -> approval gate

Error recovery flow

Purpose: define fault handling, retries, fallback, quarantine, and escalation.

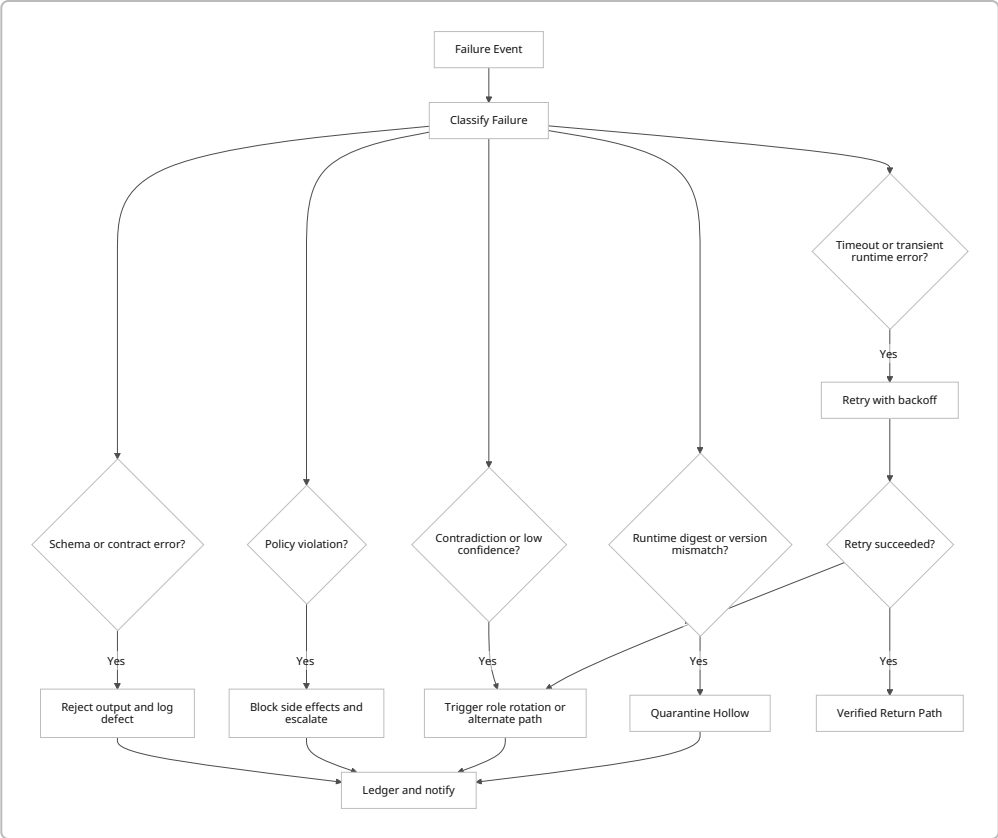
Audience: SRE, platform, security, QA.

Elements: failure classifier, schema error, timeout, nondeterminism, policy fail, contradiction, alternate Hollow, role escalation, human review, dead-letter/quarantine.

Layout notes: place classifier at center with multiple failure branches; join success branches at bottom.

Color/typography: amber for recoverable, red for quarantine, blue for reasoning escalation.

Export: SVG master, PNG 2x, PDF.



Failure -> Classify
schema error -> reject
timeout -> retry -> if fail, escalate
policy fail -> block and escalate
contradiction -> role rotation
version/digest mismatch -> quarantine
All paths -> ledger + notify

Technical paper blueprint

A strong technical paper on this architecture should present Caleb AI not as a vague “multi-agent” idea, but as a **governed execution architecture** that separates adaptive reasoning from bounded deterministic work. The outline below is optimized for a professional whitepaper, investor appendix, architecture decision record, or conference-style systems paper.

Paper section	What it should do	Original Caleb AI design or source backed
Executive summary	State the core thesis: models think, Hollows work, Caleb orchestrates, Verified Return Path certifies	Original Caleb AI design filecite turn0file0

Paper section	What it should do	Original Caleb AI design or source backed
Problem framing	Explain why monolithic model calls are expensive, hard to debug, and risky for deterministic work	Source-backed ²⁹
Architectural overview	Introduce Orchestration Core, Model API Layer, Communication Bus, Hollow Server, Verified Return Path	Mixed
Hollow model	Define Hollow properties, taxonomy, contracts, lifecycle, and chaining rules	Original Caleb AI design with source-backed parallels fileciteurlturn0file1 30
Role rotation model	Define Planner, Analyst, Critic, Synthesizer and escalation logic	Original Caleb AI design with source-backed parallels ³¹
Trust and provenance	Define Verified Return Path, trust tiers, ledger model, replay rules, audit surfaces	Mixed ³²
Security model	Cover prompt injection, insecure output handling, excessive agency, consent, least privilege, sandboxing	Source-backed ⁹
Governance and versioning	Map control ownership, SemVer discipline, approval gates, deprecation and quarantine policy	Source-backed plus original policy model ²⁴
Testing and evaluation	Define contract tests, replay tests, adversarial tests, and role-rotation benchmark triggers	Mixed ³³
Cost and performance	Show why token minimization, smaller models, deferred tool surfaces, and local execution matter	Source-backed ²⁶
Implementation roadmap	Lay out phases from typed Hollow core to governed cloud-hybrid scaling	Original Caleb AI design
Discussion and limitations	Explain where role rotation is worth it and where simpler prompting or voting may win	Source-backed ³⁴

The source priority order for the paper should be explicit. **Primary and official sources first:** OpenAI function calling, strict mode, structured outputs, and cost optimization; Anthropic tool use and building effective agents; MCP; JSON Schema; W3C PROV-O; NIST AI RMF; OWASP GenAI project; OpenTelemetry; SemVer; and Ink & Switch local-first. Those should carry the load-bearing architectural and governance claims. **Research sources second:** Toolformer, ReAct, Plan-and-Solve, Self-Refine, Reflexion, multi-agent debate, debate-versus-vote, cost-efficiency work, and workflow-engine papers. These should be used

mainly to justify when extra reasoning structure helps and when it becomes too expensive or too complex.

35

Tables and implementation roadmap

Hollow registry example

hollow_id	Class	Contract version	Deterministic	Side effects	Trust default	Owner
hollow.arithmetic	Compute	1.0.0	Strict	None	T2	Platform
hollow.schema_validate	Validation	1.1.0	Strict	None	T2	Platform
hollow.threshold_policy	Policy	1.0.0	Strict	None	T3	Governance
hollow.provenance_stamp	Provenance	1.0.0	Strict	Ledger write only	T3	Platform
hollow.context_packager	Transform	0.9.0	Bounded	None	T1	Orchestration
hollow.retry_backoff	Recovery	1.0.0	Strict	Scheduler control	T2	SRE

Hollow manifest template

Field	Type	Required	Meaning
<code>hollow_id</code>	string	yes	Stable public Hollow identifier
<code>display_name</code>	string	yes	Human-readable name
<code>description</code>	string	yes	What the Hollow does and does not do
<code>contract_version</code>	semver string	yes	API version of the Hollow
<code>input_schema_ref</code>	URI	yes	JSON Schema for inputs
<code>output_schema_ref</code>	URI	yes	JSON Schema for outputs
<code>determinism_mode</code>	enum	yes	<code>strict</code> , <code>bounded_external_state</code> , or <code>best_effort</code>
<code>allowed_side_effects</code>	array	yes	Empty for pure Hollows; explicit list otherwise
<code>required_secrets</code>	array	no	Minimum scoped credentials
<code>runtime_digest</code>	hash	yes	Runtime identity for trust and attestation
<code>owner_team</code>	string	yes	Responsible team

Field	Type	Required	Meaning
<code>policy_tier</code>	enum	yes	Side-effect and trust sensitivity tier
<code>retry_policy</code>	object	yes	Retry limits, backoff, idempotency policy
<code>observability_tags</code>	array	no	Trace, metrics, logging labels
<code>deprecation_policy</code>	object	yes	Sunset and compatibility rules

Comparison against current AI workflows

Dimension	Typical current AI workflow	Caleb AI with Hollows and role rotation	Why the Caleb AI model is stronger
Deterministic work	Model often performs math, parsing, threshold logic, and formatting inline	Deterministic work moves into Hollows	Better reliability and lower reasoning waste on logic-heavy subtasks ³⁶
Tool boundaries	Tools may be loosely defined or overexposed	Typed contracts, strict schemas, deferred exposure	Reduces malformed calls, context bloat, and token waste ³⁷
Trust model	Tool output may be trusted too early	Verified Return Path assigns trust tier before reuse	Better security and auditability ³⁸
Reasoning structure	One-pass prompt or brittle agent loop	Conditional role rotation with Planner, Analyst, Critic, Synthesizer	Better decomposition on hard tasks, but only when warranted ³⁹
Observability	Logs may be partial or disconnected	Unified trace plus provenance ledger	Better debugging and governance visibility ⁴⁰
Governance	Validation and execution often blur together	Validators and executors are distinct actors	Better alignment with NIST best practice on verification separation ⁴¹
Locality	Sensitive logic often sent to cloud by default	Local-first Hollow execution with cloud-hybrid reasoning	Better privacy, ownership, and responsiveness ¹⁰

Recommended first twelve Hollows

Hollow	Core inputs	Core outputs	Main risk	Verification strategy
Arithmetic Hollow	expression, numeric context	numeric result, proof trace	overflow or coercion bugs	property tests against reference math engine

Hollow	Core inputs	Core outputs	Main risk	Verification strategy
Unit Conversion Hollow	value, source unit, target unit	converted value	unit-map drift	golden tables and dimensional analysis tests
Date Time Logic Hollow	timestamp, timezone, calendar rule	normalized date/time result	timezone ambiguity	replay tests across DST and locale fixtures
String Regex Extraction Hollow	text, extraction rule	structured fields	overmatch or silent miss	corpus-based precision/recall fixtures
Table Normalization Hollow	CSV/JSON rows, schema	normalized table	coercion and null semantics	schema tests and canonical fixture outputs
JSON Schema Validation Hollow	candidate object, schema ref	valid/errors/annotations	false pass from schema mismatch	strict validation with JSON Schema 2020-12
Threshold Policy Hollow	metrics, rule set	allow/deny/escalate	policy brittleness	deterministic rule tests and approval review
Provenance Stamp Hollow	artifact refs, hashes, actor IDs	PROV envelope	incomplete linkage	ledger replay and referential integrity checks
Hash Signature Hollow	payload, key ref	digest/signature	key misuse	known-answer tests and key scoping checks
Context Packager Hollow	verified artifacts, token budget	model-ready evidence pack	omission bias	budget-aware packing tests and manifest diffing
Retry Backoff Hollow	failure class, retry policy	next action	retry storm	chaos testing and idempotency checks
Consensus Aggregator Hollow	multiple role outputs	agreement map, dissent map	plurality masking errors	benchmarked disagreement thresholds, replay analysis

Implementation roadmap

A practical roadmap for Caleb AI should move in layers rather than trying to ship “the full agent platform” at once.

The first phase should establish **typed Hollow contracts and the Hollow registry**. That means JSON Schema inputs and outputs, SemVer governance, trace IDs, deterministic runtime digests, and no side effects except where explicitly declared. Without that foundation, later provenance and trust features will be cosmetic. ⁴²

The second phase should implement the **Verified Return Path** and the **provenance ledger**. Caleb AI should be able to say not merely that output exists, but how it was generated, what it used, who initiated it, whether it replayed, and what policy tier it achieved. That is the point where Hollows become a trustworthy substrate rather than just local helpers. ³²

The third phase should add the **Model API Layer** and a **minimal orchestration policy**: route deterministic bounded tasks to Hollows, simple low-stakes tasks to single model passes, and only hard tasks to structured workflow patterns. That follows Anthropic's simplest-first production advice and prevents Caleb AI from overengineering itself too early. ⁶

The fourth phase should add **role rotation gating** and measure it rigorously. Planner → Analyst → Critic → Synthesizer should not be the default; it should be activated when ambiguity, task branching, contradiction risk, or consequence severity crosses a measurable threshold. Evaluation should compare single-pass, Hollow-enhanced, and role-rotated paths on both quality and cost. ⁴³

The fifth phase should add **cloud-hybrid deployment**: local-first Hollows, cloud model providers, scoped remote data access, user-consent boundaries, and different trust policies for local-only, cloud-assisted, and side-effect-capable tasks. That phase is where Caleb AI becomes portable across workstation, private server, and enterprise environments without losing its core architecture. ⁴⁴

Open questions and limitations

Some parts of this report are intentionally left as design choices rather than disguised assumptions. The brief did not specify a deployment substrate, so this report avoids locking Caleb AI to any one message broker, database, container runtime, or orchestration framework. That keeps the architecture portable, but it also means the exact implementation tradeoffs for Kafka vs NATS, SQLite vs Postgres, WASM vs containers, or Merkle log vs relational ledger are still open. This is a limitation of scope, not a gap in the core model. ^{filecite turn0file1}

The strongest evidence for role rotation's upside is real but mixed. Planner-like decomposition, evaluator loops, reflection, and debate can all improve difficult tasks, but recent work also shows that some measured gains come mostly from voting or ensembling, not from rich debate itself, and unnecessary orchestration can lose to simpler in-context designs on procedural tasks. Caleb AI should therefore treat role rotation as a benchmarked escalation mechanism rather than an ideology. ⁴⁵

Finally, any Hollow that depends on mutable external data ceases to be purely deterministic. Weather lookups, market rates, web retrieval, and live database reads are still valuable, but they should be labeled as **bounded external state** rather than **strict deterministic** Hollows, with lower default trust and stricter provenance requirements. MCP, OpenAI, and Anthropic tooling all make these external integrations possible, but their own documentation also implies the need for clearer trust boundaries, explicit consent, and careful schema discipline when those tools are used. ⁴⁶

1 4 5 12 15 17 28 30 35 37 42 <https://platform.openai.com/docs/guides/function-calling>
<https://platform.openai.com/docs/guides/function-calling>

2 29 36 <https://arxiv.org/abs/2302.04761>
<https://arxiv.org/abs/2302.04761>

3 20 31 39 43 45 <https://arxiv.org/abs/2305.04091>
<https://arxiv.org/abs/2305.04091>

6 19 25 33 <https://www.anthropic.com/engineering/building-effective-agents>
<https://www.anthropic.com/engineering/building-effective-agents>

7 22 32 <https://www.w3.org/TR/prov-o/>
<https://www.w3.org/TR/prov-o/>

8 16 21 41 <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf>
<https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf>

9 18 38 <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
<https://owasp.org/www-project-top-10-for-large-language-model-applications/>

10 44 <https://www.inkandswitch.com/essay/local-first/>
<https://www.inkandswitch.com/essay/local-first/>

11 <https://arxiv.org/abs/2204.07210>
<https://arxiv.org/abs/2204.07210>

13 23 46 <https://modelcontextprotocol.io/specification/2025-03-26>
<https://modelcontextprotocol.io/specification/2025-03-26>

14 24 <https://semver.org/>
<https://semver.org/>

26 27 <https://developers.openai.com/api/docs/guides/cost-optimization>
<https://developers.openai.com/api/docs/guides/cost-optimization>

34 <https://arxiv.org/abs/2508.17536>
<https://arxiv.org/abs/2508.17536>

40 <https://opentelemetry.io/docs/concepts/signals/traces/>
<https://opentelemetry.io/docs/concepts/signals/traces/>